

Full-Stack Interview Prep — Next.js, NestJS, Monorepo & Role Fundamentals

A second study guide, broader than `INTERVIEW_PREP.md`. That file is about *this specific submission*. This one is about the underlying technologies and the full-stack role itself — the kind of questions that get asked regardless of which take-home you did, using this project as your source of concrete examples wherever possible so every answer is backed by real code, not textbook definitions.

Structure: Next.js → NestJS → Monorepo/Turborepo → general full-stack technical questions → business/behavioral questions. Each answer favors "here's the concept, here's exactly where I used it" over reciting theory.

1. Next.js

Q: What's the difference between the App Router and the Pages Router, and why does it matter? App Router (what this project uses, Next.js 16) is built around React Server Components — components render on the server by default, ship zero JS to the client unless marked `"use client"`, and routing is file-system-based via `app/` with nested layouts. Pages Router is the older model — every page is a client-renderable module, data fetching happens in `getServerSideProps`/`getStaticProps`. The practical difference I leaned on in this project: `features/tasks/components/task-view.tsx` and similar are server components by default; `"use client"` is added only where it's actually needed — anything using TanStack Query hooks, `useState`, or event handlers (`assignee-selector.tsx`, the hooks files). That's a deliberate boundary, not a default I inherited by accident — a component only pays the client-JS cost if it needs interactivity.

Q: What is a Server Component, actually, mechanically? It's a component that renders to a serialized description on the server and is never shipped to the browser as JavaScript — no hydration, no client-side re-render, can't use hooks or browser APIs. It can be `async` and read data directly (DB calls, `fetch`) without a client-side loading state, because the HTML is already complete by the time it reaches the browser. Client Components are the familiar React model — hooks, interactivity, hydrate in the browser. The two compose: a Server Component can render a Client Component as a child, but not the reverse (a Client Component can only import another Client Component, or receive a Server Component as a `children`/slot prop it doesn't re-render).

Q: Where would you draw the server/client boundary on a real page, and why? As low in the tree as possible — push `"use client"` down to the smallest interactive leaf, not up to the whole page. In this project, the outer page/route stays a server component (layout, static

structure); the `AssigneeSelector` and `TaskActivityTimeline` are client components because they need TanStack Query hooks and local `useState` for the dropdown's open/search state — but the surrounding `TaskView` layout doesn't need to be a client component just because one of its children is. Getting this boundary right is a genuine skill: draw it too high and you lose the whole point of RSC (shipping less JS); too low and things that legitimately need shared client state get awkward to wire.

Q: How is server state handled here, and why not just use Server Components for data fetching directly (no TanStack Query)? Considered implicitly by the existing codebase's architecture — server components *could* fetch directly, but this app's data is highly interactive and mutated from the client constantly (assigning a task, changing status, adding a comment), and needs optimistic updates, cache invalidation across multiple views, and refetch-on-window-focus — all things TanStack Query is built for and raw server-side fetching isn't. So the split here is: initial page structure can be server-rendered, but anything backed by data the user actively mutates goes through `useQuery`/`useMutation`, with query keys centralized in `lib/query-keys.ts` so a mutation in one feature can't silently forget to invalidate a query read by another. `useUpdateTaskAssignee` is the clearest example: it optimistically writes into the `task` query's cache, and on settle invalidates three separate query keys (the task, its activity feed, the project's task list) because a single assignment touches all three views.

Q: What is `next/font`, and did you run into any issue with it? It self-hosts and optimizes web fonts (here, Google's Inter) at build time — no runtime request to Google Fonts, better privacy and performance than a `<link>` tag. I actually hit this directly: in the sandbox I built this submission in, outbound network access to `fonts.googleapis.com` was blocked, so `pnpm build` for the web app failed at the font-fetch step specifically — `typecheck` and `lint` both still passed clean, since that's a build-time asset-fetching failure, not a code problem. Good thing to be able to explain precisely if asked "did everything actually build" — the honest answer is "typecheck/lint/API build all passed; the web app's production build hit an environment network restriction unrelated to my changes," not a vague "yes."

Q: Why file-based routing over a config-based router? Colocation and convention over configuration — the URL structure is readable directly from the `app/` directory tree, and layouts nest naturally (a layout wrapping `/projects/[id]` doesn't need to be manually composed). Trade-off: less flexible for truly dynamic/config-driven routing, but that's not a need this kind of CRUD-heavy app has.

Q: How do you handle loading/error/empty states in this architecture? Two layers. Next.js itself supports `loading.tsx`/`error.tsx` route conventions for route-level Suspense boundaries (not heavily used in this particular project's existing structure). At the component level — the one I actually built — each data-driven component handles its own `isPending`/`isError`/empty-result states explicitly: `TaskActivityTimeline` renders a

`Skeleton` while pending, an inline error message styled with the design system's danger tokens on error, and an `EmptyState` component when the feed genuinely has zero entries — three distinct states, not a single collapsed "loading OR nothing" check.

2. NestJS

Q: What is dependency injection, and where does it actually show up in this codebase?

Nest's DI container instantiates and wires classes based on constructor parameter types, decorated with `@Injectable()`. Concretely: `TaskService`'s constructor takes `@InjectModel(Task.name)` for the Mongoose model, plus `ProjectAccessService`, `ProjectMembersService`, and `UserService` — Nest resolves and injects all of them without me writing any wiring code, as long as the providing modules are imported. This matters for testability too — in principle you could substitute a mock `ProjectMembersService` in a unit test without touching `TaskService`'s code at all, because it depends on an injected interface-shaped constructor param, not a concrete import.

Q: Walk me through Nest's module system — why did you need a new module for `task-activity`, and why did you wire it the way you did instead of the more obvious way?

Modules are Nest's unit of encapsulation — they declare what they import (dependencies), what they export (what other modules can use from them), their controllers and providers. The obvious-seeming wiring for the activity feature would be: `TaskActivityModule` needs `TaskService` (to find a task and check access) *and* `TasksModule` needs `TaskActivityService` (to write an activity row when assignment changes) — but that's a circular import, which Nest will refuse to resolve cleanly. I broke the cycle the same way the existing `CommentsModule` already does for its relationship to tasks: `TaskActivityModule` imports `TasksModule` (one-directional, for reads), and `TasksModule` registers the `TaskActivity` Mongoose schema *directly* via its own `MongooseModule.forFeature(...)` call (the same way it already registers `Comment`'s schema) so it can write activity rows without importing `TaskActivityModule` at all. Two modules can register the same schema name independently — each gets its own injectable model token, no conflict — which is what makes this work.

Q: What are Guards, Interceptors, and Pipes, and which did you use?

- **Guards** decide if a request is allowed to reach the handler at all — binary yes/no, run before the handler. `JwtAuthGuard` here, registered globally via `APP_GUARD`, checked on every route unless `@Public()` opts out.
- **Pipes** transform/validate input before it reaches the handler. Nest's global `ValidationPipe` (backed by `class-validator` DTOs) is what makes `UpdateTaskAssignedDto`'s `@IsMongoId()`/`@ValidateIf` decorators actually reject

malformed input before `TasksController` even runs — the 400 for a bad `assigneeId` happens at the pipe layer, not inside my service code.

- **Interceptors** wrap the handler, can transform the response or add cross-cutting behavior (logging, timing, response shaping). Not something I added new in this assessment — the existing `HttpExceptionFilter` (an exception filter, a related-but-distinct concept — catches thrown exceptions and shapes the error response) already standardizes error shape across the API.
- Authorization itself (`assertCanView`)/(`assertCanManage`) is *not* a guard in this codebase — it's a plain service method called explicitly inside each service method that needs it. I flagged this as an architectural risk in `ASSESSMENT_NOTES.md`: because it's convention-based rather than enforced by a guard, it's exactly the kind of check a method can forget — which is precisely what happened in the reported production bug (`updateStatus` skipped it entirely). If I had more time, this is the #1 thing I'd refactor into an actual guard.

Q: Why DTOs with `class-validator` instead of validating manually in the service?

Validation at the boundary, not scattered through business logic — a DTO class like `UpdateTaskAssigneeDto` declares its own shape and rules (`@ValidateIf` + `@IsMongoId()` to allow `null` explicitly but validate a real id when present), and Nest's `ValidationPipe` enforces it automatically before the controller method body runs at all. This keeps `TasksService.assign` free to assume its `assigneeId: Types.ObjectId | null` parameter is already well-formed — it only has to reason about *business* rules (is this a project member, is the actor privileged), not *shape* validation.

Q: How does Mongoose fit into this — why Mongoose over the native MongoDB driver, and what did you actually rely on from it? Mongoose gives schema definition, validation, typed documents, and middleware/hooks on top of the native driver — the existing codebase already commits to it (every domain has a `schemas/*.schema.ts`), so I followed that rather than mixing approaches. What I relied on directly: `@Schema()`/`@Prop()` decorators to define `TaskCounter` and `TaskActivity`, `SchemaFactory.createForClass`, and — the important one — `Model.findOneAndUpdate` with `{ $inc }` for the atomic counter, which is really a native-driver-level MongoDB guarantee that Mongoose exposes directly rather than something Mongoose adds on top. Worth being precise about that distinction if asked: the atomicity is MongoDB's, not Mongoose's; Mongoose is just the typed API I called it through.

Q: What's the shape of a typical module in this codebase, and why does that consistency matter? Controller → Service → Mongoose Model, one module per domain concept, DTOs at the boundary. `TaskActivityModule` follows this exactly — a thin controller (`TaskActivityController`, one `@Get()` handler that just extracts params and delegates), all the actual logic (access check, batched actor resolution, pagination) in `TaskActivityService`. The value of following the existing shape strictly, even where I might have done it slightly

differently in a greenfield project, is that another engineer reading `task-activity/` for the first time already knows where to look for what, because it's identical in shape to `comments/`, which they've presumably already read.

3. Monorepo — pnpm workspaces + Turborepo

Q: Why a monorepo here instead of separate repos for the API and the frontend? The strongest concrete reason in this project: `packages/shared` — enums like `TaskStatus`/`TaskPriority`/`TaskActivityType` and response types like `TaskDetail` are defined **once** and imported by both `apps/api` and `apps/web`. If the API and web app were separate repos, that package would need to be published and versioned independently (npm registry or a private one), and every type change would mean a publish-then-bump-then-install cycle before the other repo could see it. In a monorepo, I added `TaskActivityType` and extended `TaskDetail` in `packages/shared/src/{tasks,api}.ts` and both `apps/api` and `apps/web` picked it up immediately on the next build — no publish step, and `tsc`/ESLint caught any place the two apps' usage drifted out of sync with the new shared shape, in the same PR, before it could ship broken.

Q: What does pnpm workspaces actually do here, mechanically? `pnpm-workspace.yaml` declares which directories are packages (`apps/*`, `packages/*`). pnpm then symlinks internal packages into each consumer's `node_modules` instead of installing them from a registry — when `apps/api` imports `@projectflow/shared`, that's a symlink to `packages/shared`, not a copied/published artifact. One `pnpm install` at the repo root resolves and hoists dependencies for every package consistently, and `pnpm --filter @projectflow/api <script>` runs a script scoped to just that package (I used this constantly — `pnpm --filter @projectflow/api typecheck` to iterate on just the backend without waiting on the frontend build).

Q: What does Turborepo add on top of pnpm workspaces — why not just `pnpm -r run build` (recursive)? Two things pnpm's recursive runner doesn't give you: **task graph awareness** and **caching**. `turbo.json` declares that `build` depends on its package's own `^build` (upstream packages build first) — so `packages/shared` always builds before `apps/api`/`apps/web` try to import it, without me manually sequencing anything. And Turborepo caches task outputs keyed by inputs — I saw this directly: after the first `pnpm typecheck`, a second run showed `cache hit, replaying logs` and finished in under a second, because nothing relevant had changed. That matters a lot in CI (don't re-run unaffected packages' tests) and locally during iteration.

Q: What goes in `packages/shared` vs. staying inside `apps/api` or `apps/web` — how do you decide? The test I applied: does *both* sides of the API boundary need to agree on this shape?

Enums (`TaskStatus`), API response types (`TaskDetail`, `TaskActivityEntry`), and constants (`TASK_ACTIVITY_TYPES`) all live in `shared` because the backend produces them and the frontend consumes them — if they drifted, you'd get a runtime bug that TypeScript can't catch across a repo boundary but *can* catch across a monorepo package boundary. DTOs (`UpdateTaskAssigneeDto`), on the other hand, stay in `apps/api` — they're an API *input* validation concern, internal to the backend, the frontend never imports or needs to agree on the DTO class itself (just the plain object shape it accepts, which is documented, not shared as code).

Q: What's a real trade-off or downside of the monorepo approach you'd mention if asked, not just the benefits? Coupling in the other direction: because `apps/web` and `apps/api` share `packages/shared` directly, a breaking change to a shared type forces both apps to be updated (and typecheck clean) together, in the same change — you can't deploy a backend change that alters `TaskDetail`'s shape independently of the frontend picking it up, the way you could with two separately-versioned repos where the frontend pins an old package version until it's ready. For a small team building one product, that's the right trade (drift is worse than coupling); for multiple independent teams shipping on different cadences, it can become a real coordination cost. I'd name that honestly rather than present monorepos as strictly better.

Q: How would you add a new shared package to this repo — say, a shared validation-schema package? `packages/<name>/` with its own `package.json` (name `@projectflow/<name>`, following the existing `@projectflow/shared` convention), add it to `pnpm-workspace.yaml`'s glob (already covers `packages/*`, so likely no change needed), give it a `tsconfig.json` extending `packages/tsconfig/library.json` (the existing base config for library packages, vs. `nest.json`/`nextjs.json` for the apps), and reference it from consumers via `"@projectflow/<name>": "workspace:*" in their package.json.`

4. Full-stack role — general technical questions

Q: How do you approach joining a codebase you didn't write? Exactly what I did here, and I can point to it concretely: read before writing. Before touching a single file, I read every module relevant to the task — both task-related backend modules, the authorization layer, the shared types, the seed script, and the *existing test suite and its fixtures*, specifically because tests show you the conventions a README won't (how the project actually expects auth headers to be sent in a request, what a "TestUser" looks like). That investigation is what let me find the production bug's root cause with certainty on the first pass, rather than guessing and testing hypotheses one at a time.

Q: How do you decide what to test? Business rules and regressions over exhaustive coverage — coverage percentage was explicitly *not* the target here (the brief says so directly), and I

agree with that as a general principle. I test: (1) every distinct business rule as its own case, not one combined test, so a future regression points at exactly which rule broke; (2) the exact reproduction steps for any bug I fixed, so it can't silently come back; (3) edge cases that are genuinely ambiguous or easy to get subtly wrong (the no-op-assignment-doesn't-log case; the "org member without a project row" case) — not edge cases for their own sake, ones I could articulate a real failure mode for.

Q: How do you think about backward compatibility / breaking changes to an API?

Concretely from this project: I added fields to `TaskDetail` (`assignee`, `viewerCanAssignOthers`) rather than replacing or renaming anything existing — any client only reading the old fields keeps working unchanged. The one place I'd flag as a judgment call, not a purely additive change: making `{ projectId, number }` a *unique* index on an existing collection is a schema change that could, in a real production migration (not this fresh-seed scenario), fail to apply if duplicate data already existed — I'd want a migration script that checks for and resolves duplicates *before* adding the constraint, in a real rollout.

Q: How do you approach performance — do you optimize by default? No, and I can point to where I explicitly *didn't*: the scaling section of `ASSESSMENT_NOTES.md` lists caching the authorization check as a real future candidate, but I didn't add it now, because there's no evidence yet that it's actually a hot path — adding a cache invalidation problem to solve a cost you haven't measured is a bad trade. Where I *did* optimize proactively: batching actor resolution in the activity feed to avoid N+1 (a `Set` + one `findManyByIds` call instead of one query per row) — because that's not "premature," it's the difference between O(1) and O(n) queries for a pattern I already knew the shape of, with no uncertainty about whether it mattered.

Q: How do you handle a requirement that's ambiguous? Make the call, document the reasoning, keep moving — don't block on it, and don't silently guess either. Rule 3 of the assignment feature (does a regular member get to unassign their own task, given the brief only says "an authorized user") is the clean example: I picked the interpretation I could defend (symmetric with the self-assign rule), wrote out *why* in both the code comment and `ASSESSMENT_NOTES.md`, and tested both directions explicitly so the decision is visible and falsifiable, not buried.

Q: How do you approach security in a feature you're building? Enforce on the server, always — client-side checks are UX, never the actual gate. Concretely: `viewerCanAssignOthers` is computed server-side and only used by the frontend to decide what to *render*; the backend's `assign()` method enforces the real rule independently regardless of what the client claims or shows. And I look for exactly this pattern when *reviewing* code too — it's the core finding in both the bug investigation and the code-review exercise in this assessment: a mutation with a

`userId` parameter that's never actually checked against anything is a real, serious finding, not a style nitpick.

Q: Tell me about a time you found a bug that wasn't the one you were looking for. This project, directly: I was specifically tasked with fixing the reported "users can modify tasks outside their project" bug and a separate concurrency bug — two named issues. While reading the codebase to find the first one, I noticed the task-numbering index wasn't unique (`{ projectId, number }` had no uniqueness constraint), which meant even after fixing the *application-level* race condition, the database itself wouldn't have caught a regression. That wasn't reported anywhere in the brief — I found it by reading the schema while investigating something else, and fixed it as a defense-in-depth measure alongside the actual counter fix, documenting why in `ASSESSMENT_NOTES.md` rather than just silently including it.

5. Business / behavioral questions

Q: How do you prioritize when you have more work than time? This assessment is a literal, recent example: a 4-hour expected effort, 6-hour hard cap, and the brief itself says the scope is "deliberately larger" than fits — deciding what to leave alone is explicitly part of what's being evaluated. My ordering: both real bugs (security + data correctness) before any UI polish, since a broken bug fix undermines everything else regardless of how nice the frontend looks; the core business rules before the frontend niceties layered on top of them (searchable selector, but only past a member-count threshold); tests for what I built over tests for hypothetical edge cases. And I wrote the things I deliberately *didn't* do into "Known Limitations" and a prioritized "two more days" list, rather than letting scope silently shrink — the interviewer can see exactly what I traded off and why, which I think is a stronger signal than trying to cram in everything.

Q: Tell me about a technical decision you had to defend or justify. The atomic-counter approach for task numbering, over the alternatives: retry-on-unique-conflict (extra round trips under contention), a distributed lock (solves a problem MongoDB already solves for free at the single-document level — real overengineering for this problem's actual shape), or dropping human-readable numbering entirely (rejected outright — it's a real, visible product feature, not just internal plumbing). I wrote the alternatives-considered reasoning explicitly into `ASSESSMENT_NOTES.md` rather than just stating the final choice, because "why not X" is usually the more interesting question in review, and I'd rather have already answered it than be asked and improvise.

Q: How do you communicate a limitation or a gap in your own work? Directly and specifically, before being asked — not vaguely, not buried. Concrete example from this exact project: I could not execute the four new backend test files in the sandbox I authored this submission in, because the MongoDB test binary needed by `mongodb-memory-server` is hosted somewhere that environment's network policy blocks. I said exactly that in `AI_LOG.md`

— what's verified (typecheck, lint, hand-traced logic against the actual service code) and what isn't (execution) — rather than implying the tests pass because I wrote them carefully. I'd rather a reviewer trust every other claim in the submission because this one was stated plainly, than have them wonder what else might be overstated.

Q: Tell me about a time you had to make a trade-off between "doing it right" and "shipping."

The activity-write consistency gap is a live example I can point to: in `TaskService.assign`, the task update and the activity-row insert are two sequential MongoDB operations, not wrapped in a transaction. "Doing it right" in the strictest sense might mean adding a multi-document transaction so the two writes are atomic. I chose not to, because this repo has no existing transaction usage, transactions require a replica set (an infrastructure requirement, not just a code change), and the actual failure window is a process crash inside a few milliseconds of sequential calls — vanishingly rare, and the failure mode if it does happen (a missing activity row, not corrupted state) is recoverable, not catastrophic. I'd rather name that trade-off explicitly than either silently ship the gap or over-invest in closing a very unlikely failure mode at the cost of introducing the first transaction into a codebase that's never needed one.

Q: How do you use AI tools in your day-to-day work, and how do you make sure you stay accountable for what they produce? The same way I used it for this assessment, which is a genuine representative example: as an active collaborator with real repo context, not a black box I paste output from. The discipline that keeps me accountable is being able to explain *why*, not just *what* — for every non-trivial decision in this submission I can point to the specific reasoning (why `TaskActivityModule` is wired the way it is to avoid a circular dependency; why I rejected the first version of the assignee selector once I actually tested the "searchable" requirement against Radix `Select`'s real capabilities). If I can't explain a piece of generated code's reasoning as if I'd written it myself, I don't ship it without first genuinely working through it until I can.

Q: How do you handle disagreement with a code reviewer or teammate? I try to separate "this is objectively wrong" from "this is a judgment call I'd defend." The code-review exercise in this assessment is a good model for how I'd want that conversation to go in real life: I didn't just list what to change, I explained *why* each finding mattered (the missing authorization check isn't a style issue, it's the same class of bug as a real production incident) and was explicit about what I *wouldn't* ask the engineer to change (the overall function shape is fine — don't rewrite for its own sake). If someone pushed back on my Rule 3 interpretation in this same project, I'd want to hear their reading of "authorized" and be genuinely willing to change my implementation if their argument was better — the goal is the right outcome, not winning the review.

Q: Why do you want to work here / what interests you about this role? (Company-specific — no project-grounded answer to script here. Prepare this one separately, tailored to the actual

company and role, before the interview. Don't leave it blank.)

Q: Do you have any questions for us? Always have 2-3 ready. Given this assessment's own emphasis on judgment and avoiding overengineering, good genuine questions in that spirit: "How does the team decide when a refactor like the authorization-guard one I flagged is worth doing, versus living with the convention-based approach?" / "What does the review process look like for a change that touches a shared package multiple apps depend on?" / "How is technical debt tracked and prioritized here day to day?"